

Scaling Laws for Neural Language Models

A compact framework for predicting language-model loss from parameters, data, and optimized compute.

MAIN VARIABLES

N, D, C_{min}

Non-embedding parameters, dataset tokens, and optimized compute explain most loss variation.

TRAINING RULE

Bigger model, fewer steps

Compute-efficient runs scale model size much faster than serial optimization steps.

DATA VIEW

Tokens become a frontier

Data quality work should ask whether curation shifts the loss frontier, not only whether it adds more text.

What question is the paper answering?

- Given a target compute budget, should we spend it on more parameters, more unique data, larger batches, or longer training?
- The paper studies many language-model runs and fits loss as a function of scale instead of treating every run as an isolated experiment.
- For data quality research, it gives a null model: a curated dataset should beat the frontier expected from token count alone.

Core prediction target

held-out cross-entropy loss

The metric is smooth and scalable, but it is not a complete data quality measure: it misses toxicity, factuality, provenance, and contamination.

Experimental setup: controlled scaling over web text

TRAINING CORPUS

WebText2

About 20.3M documents, 96GB text, 2.29×10^{10} BPE tokens, and 6.6×10^8 held-out tokens.

MODELS

768 to 1.5B

Decoder-only Transformers by non-embedding parameter count; LSTMs and recurrent variants are compared.

EVALUATION

5 text sources

WebText2 plus Books Corpus, Common Crawl, English Wikipedia, and Internet Books.

Training compute estimate: $C \approx 6 N B S$

Important detail

The paper uses non-embedding parameters because vocabulary embeddings distort small-model scaling.

This matters for reproducibility: the exact tokenization and vocabulary affect constants such as N_c and D_c .

Three basic laws: parameters, data, compute

MODEL SIZE

$$\alpha_N = 0.076$$

$L(N) = (N_c / N)^{\alpha_N}$. Doubling parameters gives a small but predictable loss decrease.

DATASET SIZE

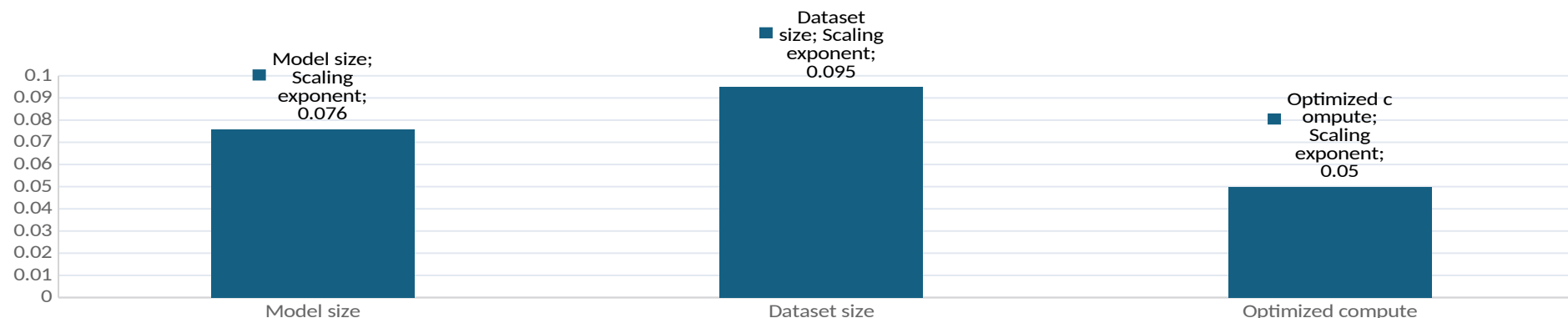
$$\alpha_D = 0.095$$

$L(D) = (D_c / D)^{\alpha_D}$. More unique tokens reduce loss, also with diminishing returns.

OPTIMIZED COMPUTE

$$\alpha_{Cmin} = 0.050$$

$L(Cmin) = (C_{cmin} / Cmin)^{\alpha_{Cmin}}$. Compute scaling is slower because compute must be split across variables.



Joint model-data law: finite data becomes measurable

$$L(N,D) = [(N_c/N)^{(\alpha_N/\alpha_D)} + (D_c/D)]^{\alpha_D}$$

Avoiding overfitting: $D \geq 5e3 * N^{0.74}$

Interpretation

- The formula separates finite-model and finite-data bottlenecks.
- Data needs grow sublinearly with model size in this setup.

Data-quality translation

Token count is only the first proxy. Deduplication, domain coverage, and information density change how many tokens are effectively new.

A good curation method should lower the loss at fixed N, D, and C.

Training dynamics: steps and critical batch size

$$L(N, S_{\min}) = (N_c/N)^{\alpha_N} + (S_c/S_{\min})^{\alpha_S}$$

$$B_{\text{crit}}(L) = B^* / L^{(1/\alpha_B)}$$

- This correction matters because runs trained at different batch sizes are not directly comparable.
- It supports the paper's claim that large models trained short can be more compute-efficient than small models trained to convergence.

ALPHA_S

0.76

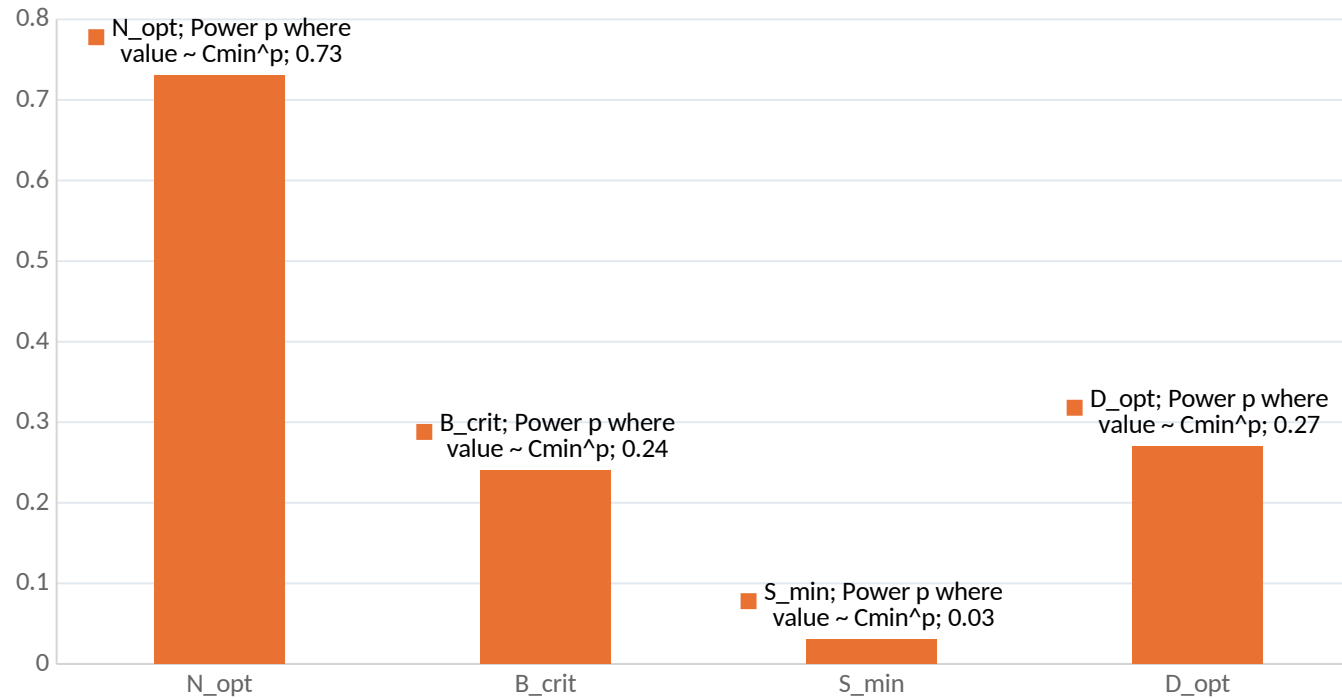
Training-step bottlenecks are much steeper than parameter or dataset exponents.

ALPHA_B

0.21

Critical batch size grows as the model reaches lower loss.

Compute-efficient allocation: what grows when compute grows?



Takeaway

When compute increases by 10x, the model should grow by about 5x, while minimum serial steps barely increase.

This is the paper's clearest operational recipe.

What does this mean for data quality research?

METRIC BASELINE

loss frontier

Ask whether a filter or selector beats the expected loss at fixed N, D, and C.

DATA SIGNAL

effective tokens

Replace raw token count with uniqueness, domain coverage, and knowledge density where possible.

RISK SURFACE

missing qualities

Cross-entropy does not measure toxicity, factuality, privacy, bias, or contamination.

- Use scaling laws as the null hypothesis for pretraining data interventions.
- Report whether a data method changes the exponent, shifts the intercept, or only changes downstream behavior.
- Pair loss with quality metrics: toxicity detection and repair, semantic consistency, knowledge density, deduplication, and contamination checks.

Limitations and follow-up experiments

- Exact reproduction is limited: WebText2, run logs, checkpoints, and official code are not released.
- Largest empirical models are much smaller than modern frontier models, so far-scale conclusions are extrapolations.
- The paper does not audit data provenance, toxicity, benchmark leakage, multilingual coverage, or factual density.
- Run a modern study where curated, random, deduplicated, and domain-balanced corpora are compared at fixed N, D, and C.

Recommended repository action

For every future paper note, record whether the work changes token quantity, effective token quality, selection policy, or evaluation reliability.